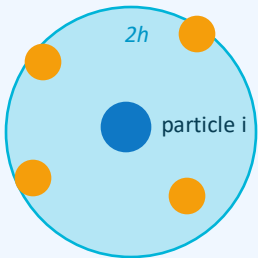


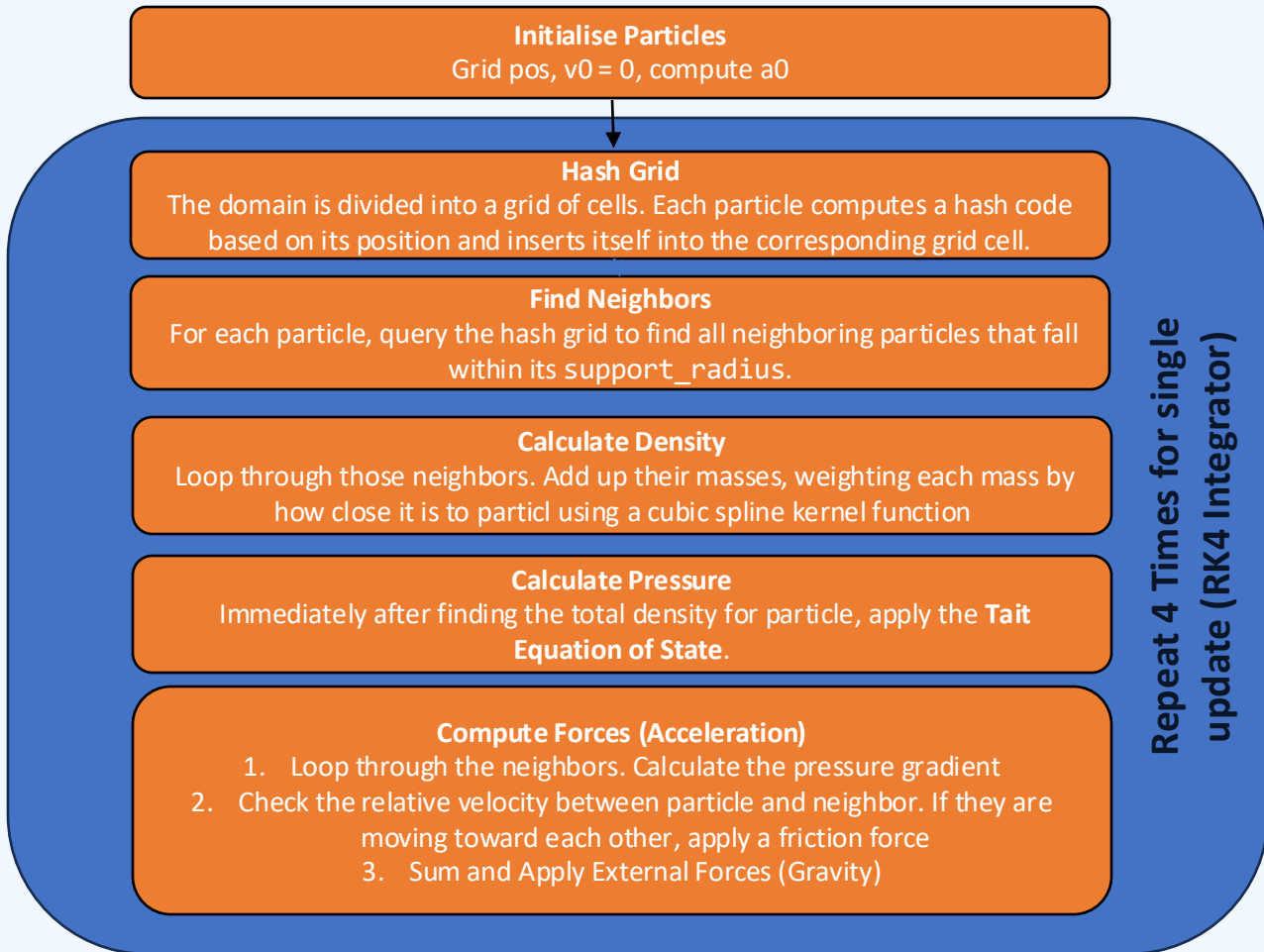
SPH Optimized Algorithm



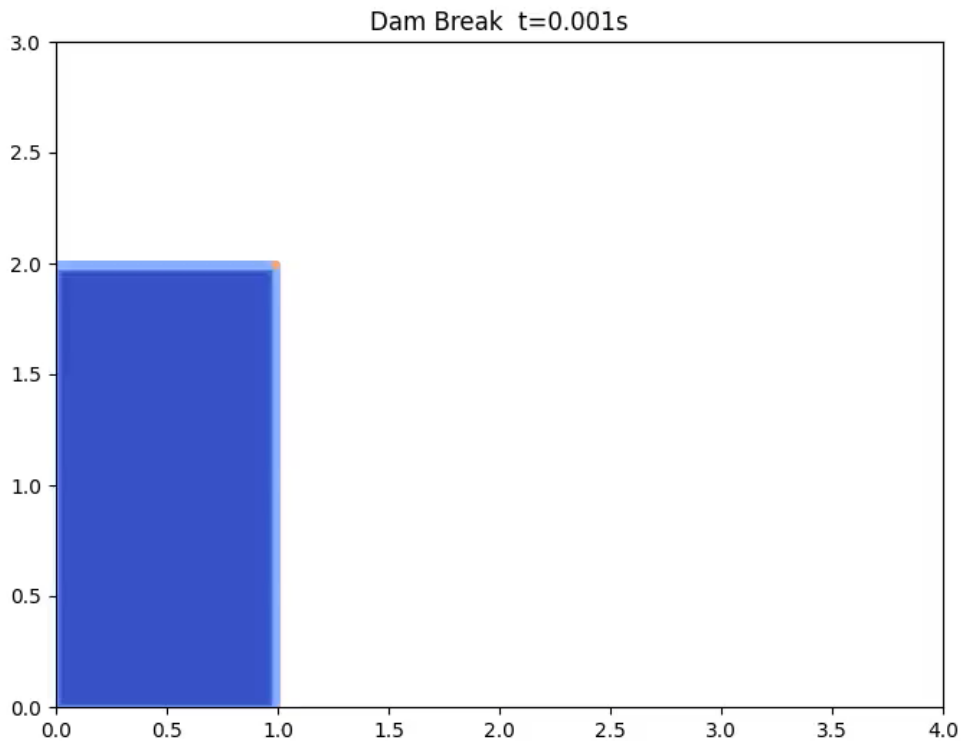
Kernels in SPH

Kernels are a way to do weighted average based on distance from a particle to compute physical quantities like density and pressure.

Here we use a cubic spline kernel



The Dam Break Problem



Simulation Setup

Domain	20.0×15.0 m
--------	--

Fluid density ρ_0	1000 kg/m³
------------------------	---

Speed of sound c_0	20 m/s
----------------------	----------------------------

No of Particles	$500,000$
-----------------	-----------------------------

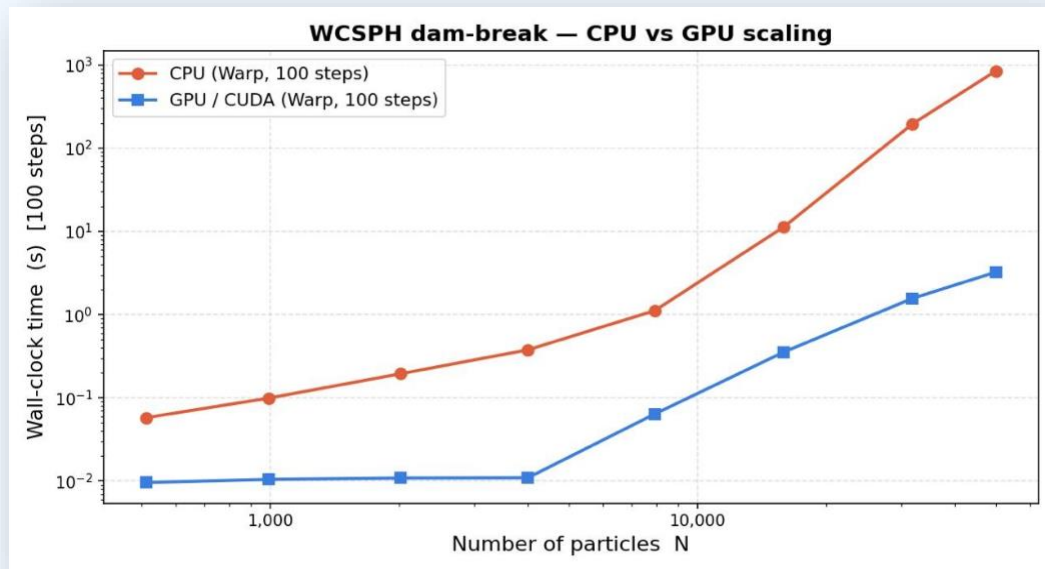
Gravity g	9.81 m/s²
-------------	--

Timestep dt	$1e-4$ s
---------------	----------------------------

Sim duration	5.0 s
--------------	---------------------------

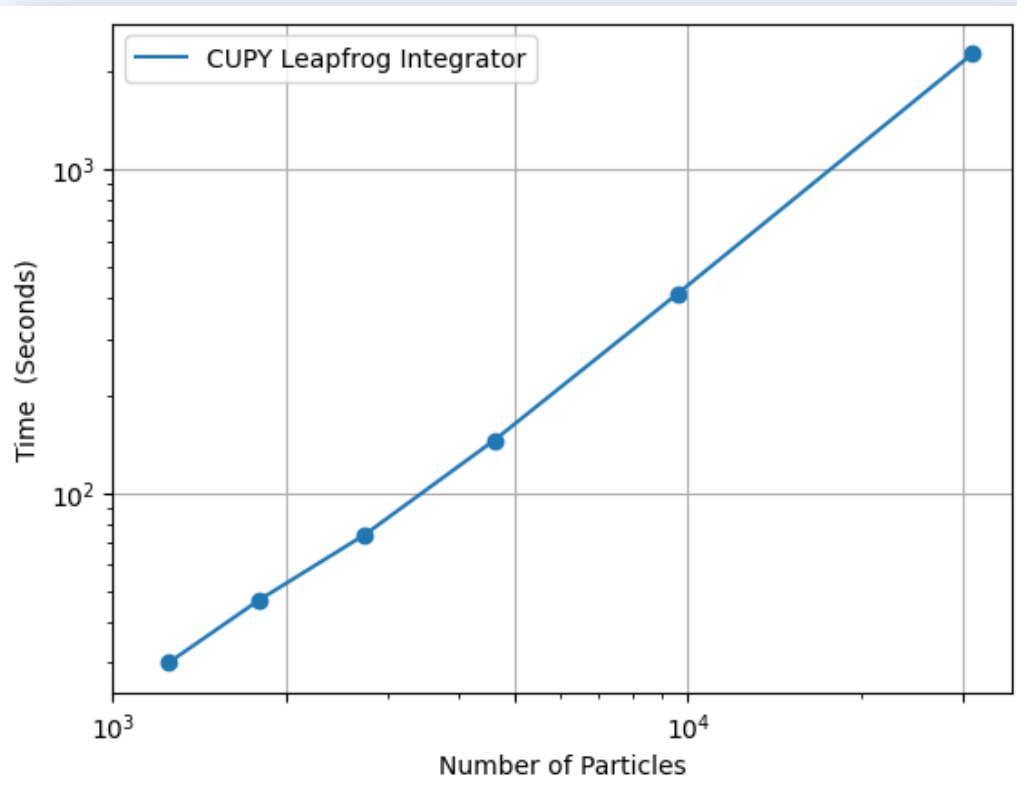
SPEED UPS

Upto 50,000 Particles



- The plot shows wall-clock time vs number of particles for a WCSPH simulation on CPU and GPU (log scale on time).
- For small particle counts, both CPU and GPU are fast, but the GPU already has slightly lower execution time.
- As N increases, CPU time grows very rapidly (roughly superlinear), reaching hundreds of seconds.
- GPU time scales much better, increasing more slowly and staying far below CPU time.
- Overall, the GPU provides a large speedup for large simulations, making it far more scalable.

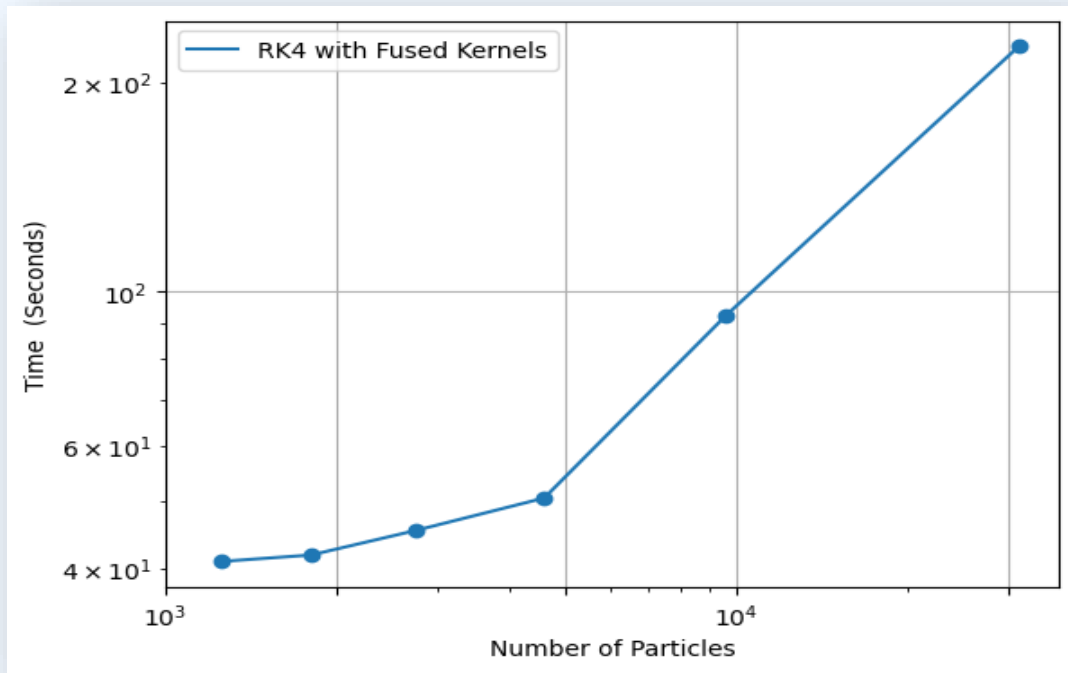
CUPY Implementation



- The graph shows how computation time scales with the number of particles for the **CUPY Leapfrog Integrator** on a log-log plot.
- There is a clear **monotonic increase**: as particle count grows ($\approx 10^3 \rightarrow 2 \times 10^4$), runtime increases significantly (tens $\rightarrow$ thousands of seconds).
- The trend appears roughly **linear on a log-log scale**, suggesting **near power-law scaling** (likely close to $O(N)$ or slightly worse).
- The slope indicates that doubling/tripling particles leads to a **more-than-proportional increase in time**, hinting at growing computational overhead.
- Overall, the graph highlights **scalability limitations**—while GPU acceleration (CuPy) helps, large particle systems still incur steep time costs.

SCALING IN OUR BEST CODE (RK4 with Kernel Fusion)

Results for Full 5 second simulation



- The graph shows execution time vs number of particles for an RK4 method with fused kernels.
- At lower particle counts ($\sim 10^3$), the runtime is around 40–45 seconds and increases slowly.
- As the number of particles grows, the time begins to rise more noticeably.
- Beyond $\sim 10^4$ particles, the runtime increases steeply, indicating higher computational cost.
- Overall, the method scales reasonably at small sizes but becomes significantly more expensive for large particle counts.

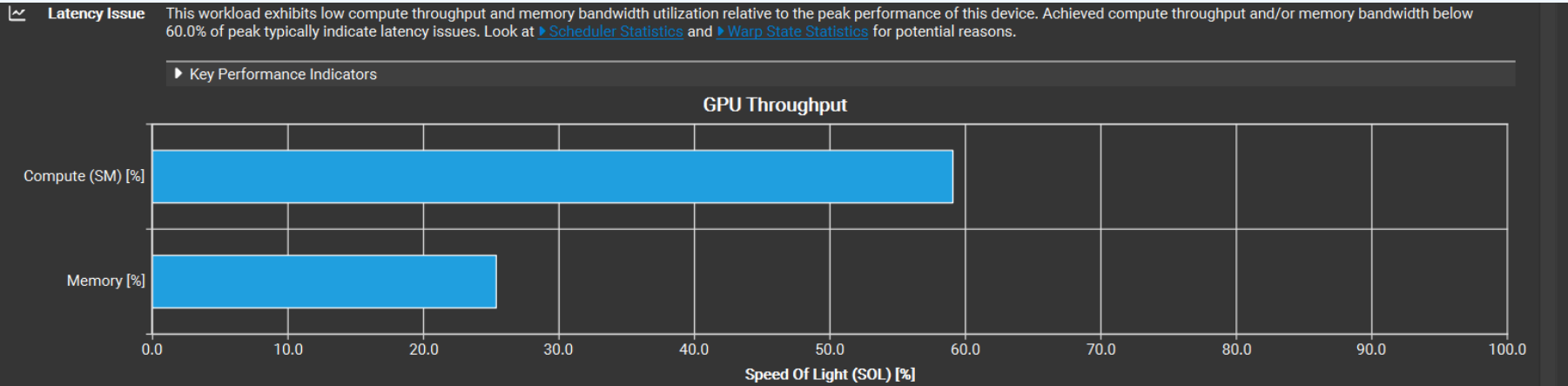
Warp Implementation (Nsight Profiler Results)

Time	Total Time	Instances	Avg Time	Kernel Name
64.20%	8.164 s	2358	3.462 ms	acceleration_neighbors_kernel
24.90%	3.167 s	2358	1.343 ms	_density_pressure_neighbors_kernel
2.10%	268.917 ms	1769	152.016 μ s	_rk4_prepare_stage_kernel
1.20%	153.437 ms	589	260.504 μ s	rk4_finalize_kernel
1.10%	133.911 ms	2359	56.765 μ s	Pack Position Kernel
0.40%	51.859 ms	589	88.046 μ s	Boundary Kernel

WARP Implementation

Initial Warp Kernels

Acceleration Kernel

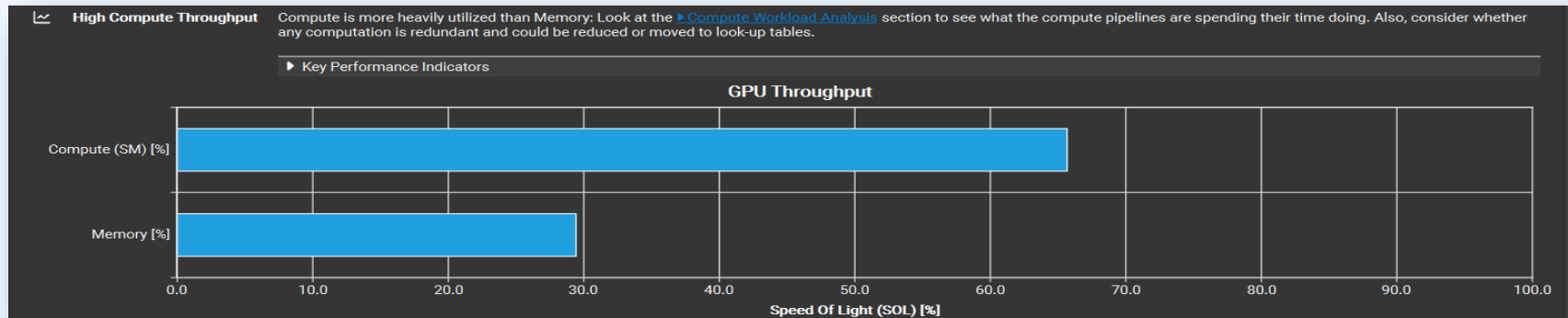


Problems

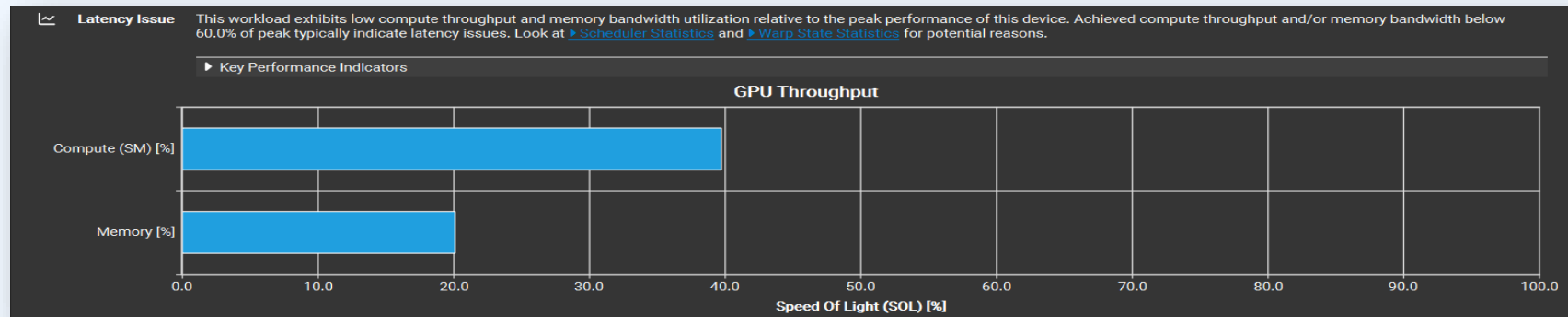
1. The algorithm launches separate kernels in intermediate stages. These could be fused to increase computation done per single memory load
2. The kernels have partial waves which should have integer number of waves

WARP Implementation

Density Kernel

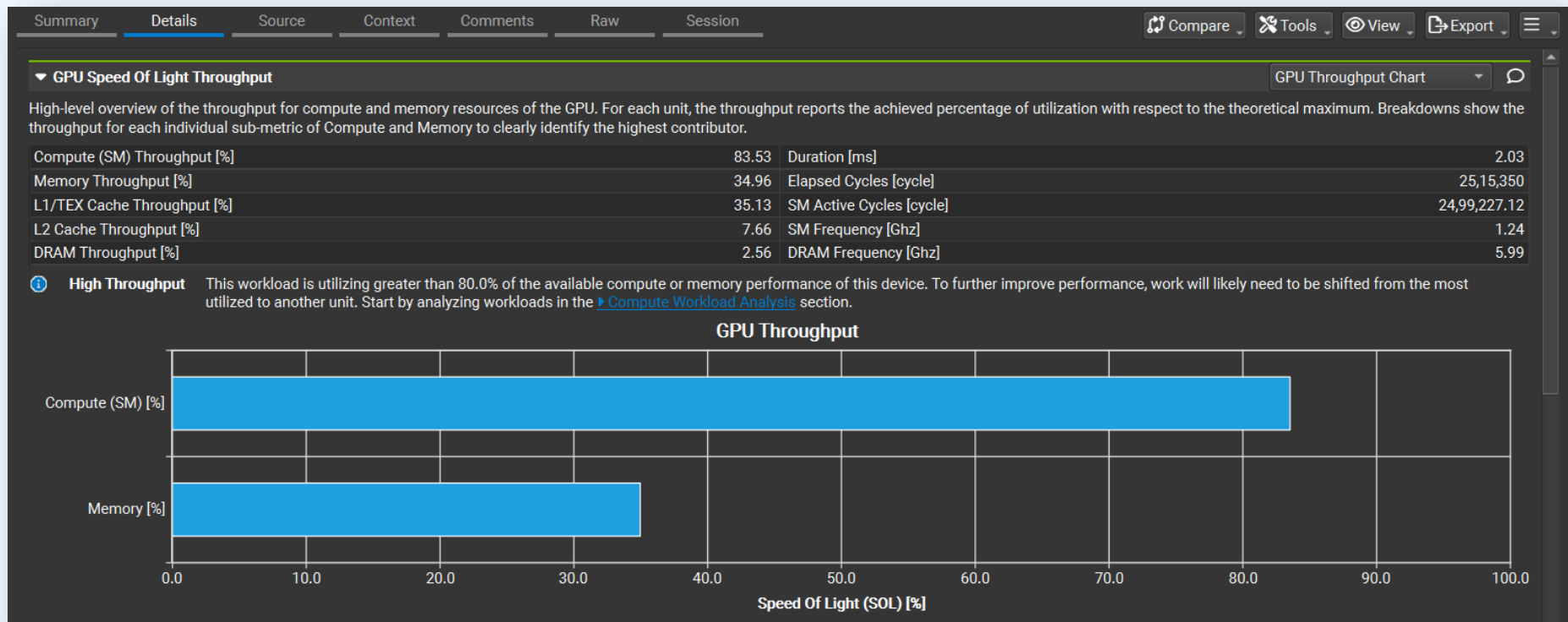


Pressure Kernel



WARP Implementation

Density Kernel + Pressure Kernel (Fused)



WARP Implementation

Kernel Fusion: Fused copy_vec and combine_state_kernel

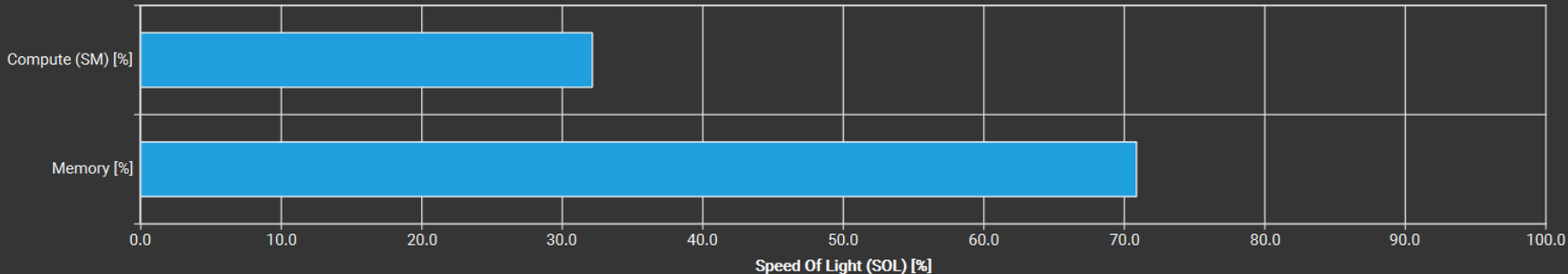


High Memory Throughput

Memory is more heavily utilized than Compute: Look at the [Memory Workload Analysis](#) section to identify the DRAM bottleneck. Check memory replay (coalescing) metrics to make sure you're efficiently utilizing the bytes transferred. Also consider whether it is possible to do more work per memory access (kernel fusion) or whether there are values you can (re)compute.

Key Performance Indicators

GPU Throughput



SCOPE OF IMPROVEMENT

NVIDIA RTX PRO 600

Current Particle Limit (For fixed time step): 500,000

Block Size: 256

Total Number of Block Required: $500,000/256 = 1954$

Number of SMs in NVIDIA RTX PRO 600: $188*4 = 752$

Number of Waves Required: $1954/752 = 2.60$

Nsight has confirmed these results

NVIDIA RTX 3050 (Laptop)

Current Particle Limit (For fixed time step): 500,000

Block Size: 256

Total Number of Block Required: $500,000/256 = 1954$

Number of SMs = 16

Number of Waves Required: $1954/752 = 30.5$

Nsight has confirmed these results

Grid Size	1,954
Cluster Size	0
Cluster Scheduling Policy	PolicySpread
Block Size	256
Threads [thread]	5,00,224
Waves Per SM	2.60
Uses Green Context	0
# SMs [SM]	188
Enabled TPC IDs	all

We could attempt to reduce the tail wave of 0.6 to increase efficiency either by decreasing or increasing the number of particles, so there are either 2 or 3 waves. This will lead to 33% increase in performance

► Launch Statistics	
Summary of the configuration used to launch the kernel. The launch configuration defines the size of the kernel grid. The launch configuration maximizes device utilization.	
Grid Size	1,954
Registers Per Thread [register/thread]	59
Block Size	256
Threads [thread]	5,00,224
Waves Per SM	30.53
Uses Green Context	0
# SMs [SM]	16
Enabled TPC IDs	all

Here optimizing the tail wave will not have much effect as its contribution is only 0.5/30 compared to 0.6/2 (33%) earlier